

DAY 12 — CONVERSATION MEMORY + QUERY REWRITER + MULTI-TURN ENGINEERING COPILOT

✓ GOAL

Move from:

single-turn QA ✓

to:

multi-turn engineering copilot ✓

that supports:

- ✓ follow-up questions
 - ✓ conversational memory
 - ✓ contextual retrieval
 - ✓ engineering discussion continuity
 - ✓ report summarization
 - ✓ conversational RAG
-

✓ WHY DAY 12 IS IMPORTANT

Current system:

Q1 → retrieve → answer

BUT follow-up questions fail:

Q1: What is front pad maximum temperature?

Q2: Compare with rear liner temperature.

because retrieval only sees:

Compare with rear liner temperature

without previous context.

✓ DAY 12 FIXES

- ✓ session memory
- ✓ query rewriting
- ✓ context-aware retrieval

- ✓ multi-turn reasoning
 - ✓ engineering workflow continuity
-

✓ DAY 12 ARCHITECTURE

User Question

↓

Conversation Memory ✓

↓

Query Rewriter ✓

↓

Router ✓

↓

RAG / SQL Retrieval ✓

↓

Context Builder ✓

↓

Qwen2.5 ✓

↓

Store Conversation ✓

✓ TEMPLATE STRUCTURE

app/

└─ memory/

| └─ session_memory.py ✓

| └─ query_rewriter.py ✓

|

└─ agent/

| └─ ask.py ✓ MODIFY

```
|
|
|— summarizer/
| |— report_summary.py ✓
|
|
|— tests/
| |— test_summary.py ✓
```

✓ STEP 1 — SESSION MEMORY

 **app/memory/session_memory.py**

```
conversation_history = []
```

```
def add_message(role, content):
```

```
    conversation_history.append({
        "role": role,
        "content": content
    })
```

```
def get_history():
```

```
    return conversation_history
```

```
def clear_history():
```

```
    conversation_history.clear()
```

✓ STEP 2 — QUERY REWRITER

 **app/memory/query_rewriter.py**

```
from app.memory.session_memory import (  
    get_history  
)
```

```
def rewrite_query(current_query):
```

```
    history = get_history()
```

```
    # ✅ previous user queries
```

```
    previous_queries = [  
  
        h["content"]  
  
        for h in history  
  
        if h["role"] == "user"  
    ]
```

```
    # ✅ no history
```

```
    if not previous_queries:
```

```
        return current_query
```

```
    previous_query = previous_queries[-1]
```

```
    # ✅ avoid duplicate expansion
```

```
    if current_query.lower() in previous_query.lower():
```

```
        return current_query
```

 contextual rewrite

```
rewritten = (  
    previous_query  
    + "  
    + current_query  
)
```

```
return rewritten
```

STEP 3 — MODIFY ask.py

 **app/agent/ask.py**

```
import ollama
```

```
from app.agent.graph import (  
    run_agent  
)
```

```
from app.memory.session_memory import (  
    add_message  
)
```

```
from app.memory.query_rewriter import (  
    rewrite_query  
)
```

```
def ask_question(query):
```

```
#  store user query
```

```
add_message(  
    "user",  
    query  
)
```

```
#  rewrite using conversation
```

```
enhanced_query = rewrite_query(  
    query  
)
```

```
#  retrieval
```

```
tool_output = run_agent(  
    enhanced_query  
)
```

```
context = ""
```

```
for r in tool_output["results"]:
```

```
    if isinstance(r, dict):
```

```
        context += (  
            str(  
                r.get("document", "")  
            )  
            + "\n"  
        )
```

else:

```
context += (  
    str(r)  
    + "\n"  
)
```

```
prompt = f"""
```

You are an automotive engineering assistant.

Answer ONLY from the provided context.

If answer is unavailable say:

"Answer not found in context."

=====

CONTEXT

=====

{context}

=====

QUESTION

=====

{query}

"""

```

response = ollama.chat(

    model="qwen2.5",

    messages=[

        {

            "role": "user",

            "content": prompt

        }

    ]

)

answer = response["message"]["content"]

# ✅ store assistant response
add_message(

    "assistant",

    answer

)

return answer

```

✅ STEP 4 — REPORT SUMMARIZER

 **app/summarizer/report_summary.py**

```

import ollama

from app.query.sql_query import (

    run_sql

```


)

```
def summarize_report():
```

```
    rows = run_sql("""
```

```
        SELECT content
```

```
        FROM engineering_chunks
```

```
        LIMIT 100
```

```
    """)
```

```
    combined = "\n".join(
```

```
        r[0]
```

```
        for r in rows
```

```
    )
```

```
    prompt = f"""
```

Summarize this automotive engineering report.

Focus on:

- brake system
- temperatures
- key observations
- important values
- testing conditions

Report Content:

```
{combined}
```

```
"""
```

```
response = ollama.chat(
```

```
    model="qwen2.5",
```

```
    messages=[
```

```
        {
```

```
            "role": "user",
```

```
            "content": prompt
```

```
        }
```

```
    ]
```

```
)
```

```
return response["message"]["content"]
```

✅ STEP 5 — UPDATE main.py

 **main.py**

```
import os
```

```
from app.ingestion.pipeline import (
```

```
    process_pdf
```

```
)
```

```
from app.agent.ask import (
```

```
    ask_question
```

```
)
```

```
from app.memory.session_memory import (  
    clear_history  
)
```

```
from app.summarizer.report_summary import (  
    summarize_report  
)
```

```
PDF_PATH = r"D:\new_train\training\Pdf_praser_llm\data\0183-VP-TDM-06-08196-C006-1339-BRAKE  
THERMAL MAPPING CITY CONDITION Vehicle Dynamics.pdf"
```

```
# ✅ ingest once
```

```
if not os.path.exists("chroma_db"):
```

```
    print("\n✅ First-time ingestion started...\n")
```

```
    process_pdf(PDF_PATH)
```

```
    print("\n✅ Ingestion completed.\n")
```

```
else:
```

```
    print("\n✅ Existing DB found. Skipping ingestion.\n")
```

```
# ✅ chat loop
```

```
while True:
```

```
    query = input("\nAsk: ")
```

```
if query.lower() == "exit":
```

```
    break
```

```
#  clear memory
```

```
if query.lower() == "clear":
```

```
    clear_history()
```

```
    print("\n  Memory cleared.\n")
```

```
    continue
```

```
#  summarize report
```

```
if query.lower() == "summary":
```

```
    summary = summarize_report()
```

```
    print("\n=== SUMMARY ===\n")
```

```
    print(summary)
```

```
    continue
```

```
response = ask_question(query)
```

```
print("\n=== RESPONSE ===\n")
```

```
print(response)
```

✅ **EXAMPLE MULTI-TURN FLOW**

USER

What is front pad maximum temperature?

ASSISTANT

163, 179, 188 deg C

USER

Compare with rear liner temperature

INTERNAL QUERY REWRITE

What is front pad maximum temperature?

Compare with rear liner temperature

ASSISTANT

Front pad maximum temperatures:

163, 179, 188 deg C

Rear liner maximum temperatures:

153, 153, 146 deg C

✅ **WHAT DAY 12 ACHIEVES**

✅ conversational memory

✅ follow-up reasoning

✅ contextual retrieval

✅ engineering dialogue

✓ report summarization

✓ multi-turn RAG

✓ query rewriting

✓ **CURRENT SYSTEM**

PDF

↓

Extraction

↓

Normalization

↓

Filtering

↓

Embeddings

↓

ChromaDB

↓

DuckDB

↓

Hybrid Retrieval

↓

Agent Routing

↓

Conversation Memory ✓

↓

Query Rewriter ✓

↓

Qwen2.5 ✓

✅ **YOU ARE NOW HERE**

✅ DAY 12 COMPLETE

Next:

DAY 13

→ engineering workflow agents

→ anomaly detection

→ recommendation engine

→ comparative analysis

→ autonomous report insights